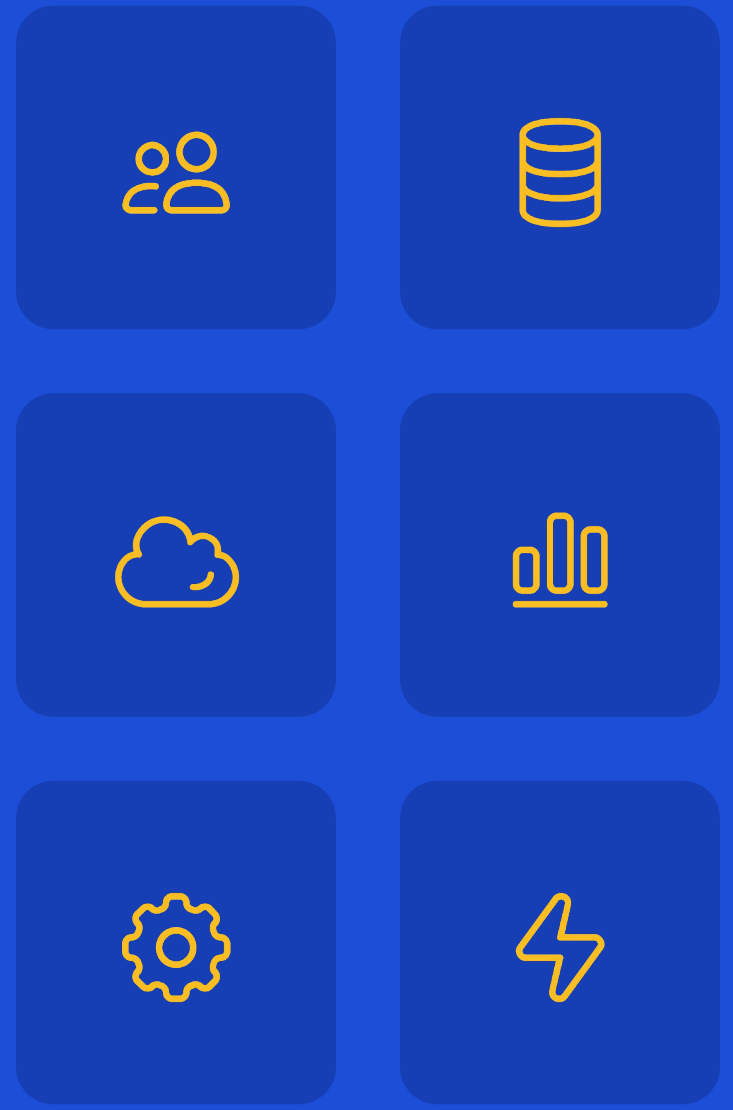
ARCHITECTURE DES SYSTÈMES D'ENTREPRISE · CHAPITRE 1

Introduction à l'architecture d'un système d'entreprise



3ème Licence en Sciences d'Informatique · 2025-2026

Institut Supérieur d'Informatique et de Mathématiques de Monastir

Enseignante : Dr. Aljia Bouzidi

Objectifs du chapitre

1. **Appréhender**
   Un système d'entreprise à haut niveau, dans sa globalité

2. **Comprendre l'architecture**
   Celle d'un système d'entreprise, et l'architecture logicielle

3. **Comprendre le cycle de vie**
   Celui d'un logiciel, de la perception du besoin à la livraison

4. **Comprendre le SDLC**
   Le cycle de développement logiciel et ses activités



Fil rouge : de l'entreprise et de ses besoins jusqu'au logiciel livré.

Plan du chapitre

1

Le système d'entreprise

Sous-systèmes et enjeux

2

Architecture d'un système d'entreprise

ESA et architecture logicielle

3

Mise en œuvre d'un système d'entreprise

Classique, unifié, agile

4

Activités de développement

SDLC, de l'étude préalable à l'implémentation

📖 **Annexes : diagrammes UML · MCD et MLD · exercice et révision**

QUESTION D'ACCROCHE

Vous commandez un repas sur une application : combien de systèmes interviennent ?

- Application mobile
- Gestion de la relation client (GRC)
- Paiement en ligne
- Stock et menu du restaurant
- Suivi de la livraison
- Comptabilité et facturation

Une entreprise est un ensemble de systèmes en interaction.

CAS D'ÉTUDE

Notre cas : une agence de location

Des clients réservent une voiture pour des dates précises ; des agents tiennent le comptoir ; la banque valide le paiement.

PARTIE 1

Le système

Qui fait quoi dans l'agence : comptoir, fiches, direction ?

PARTIE 2

L'architecture

Comment organiser le logiciel de réservation ?

PARTIE 3

La méthode

Comment s'organiser pour le construire ?

PARTIE 4

Les étapes

Quels modèles dessiner : cas d'utilisation, classes, tables ?

Partie 1

Le système d’entreprise

Analyse systémique · Sous-systèmes · Enjeux de l’informatisation · SI et système informatique

L’entreprise vue comme un système

DÉFINITION D’UN SYSTÈME

« Ensemble d’éléments en interaction dynamique, organisé en fonction d’un but »

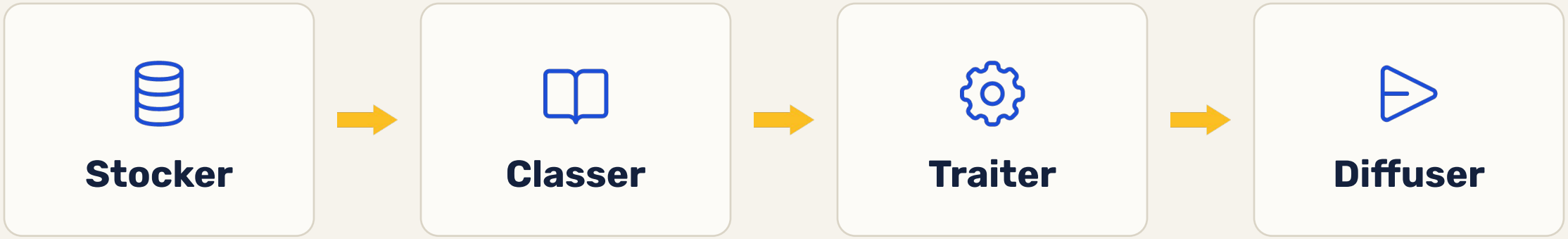
Analyse systémique, apparue dans les années 1970

- **Moyens humains**
- **Moyens matériels**
- **Moyens financiers**
- **Moyens techniques**

Toute organisation humaine (l’État, une famille…) peut être perçue comme un système.

Le système d’entreprise (SE)

Ensemble des **ressources** (personnel, matériel, logiciel…) qui participent, au sein de l’entreprise, à quatre opérations sur l’information :



Trois sous-systèmes en interaction

SP

Système de pilotage

ou système de décision

- **Réfléchir** : adaptation, conception
- **Décider** : prévoir, planifier
- **Contrôler** : la qualité

SI

Système d'information

la mémoire et le lien

- Générer des informations
- Mémoriser
- Diffuser
- Traiter

SO

Système opérant

le cœur de métier

- Transformer
- Produire

Un même employé peut être acteur de chacun des trois sous-systèmes.

Comment les sous-systèmes communiquent



Produit l'information élémentaire, exécute les ordres

Collecte et traite l'information, la transforme en information élaborée

Décide à partir de l'information élaborée

Le système d'information : un outil de communication interne.

EN CLAIR

Une pizzeria, trois sous-systèmes

SP · PILOTAGE

Le gérant

Choisit le menu, fixe les prix, décide quelle quantité de farine commander.

SI · INFORMATION

Caisse et bons

Bons de commande, caisse, stock, ventes du jour : l'information circule.

SO · OPÉRANT

Cuisine et livraison

Prépare la pâte, cuit les pizzas, livre les clients.

Le SI relie les deux : il remonte les ventes au gérant, et redescend ses décisions à la cuisine.

QUIZ · 1 MIN

Un directeur ouvre une agence à l'aide d'un tableau de bord. Qui décide ?

- **A** Le système opérant
- **B** Le système d'information
- **C** Le système de pilotage
- **D** Aucun des trois

Réponse C : le système de pilotage

Le SI fournit le tableau de bord, mais la décision relève du pilotage (réfléchir, décider, contrôler).

Pourquoi informatiser le SI ?

Productivité
Améliorer l'efficacité des utilisateurs

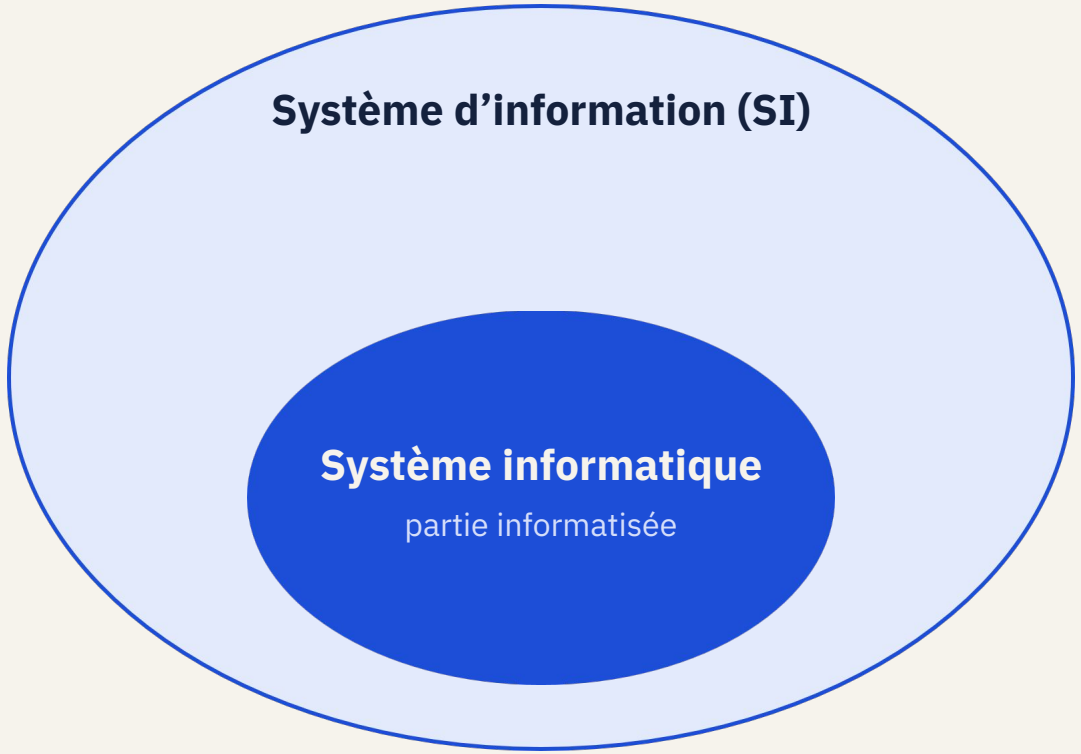
Conditions de travail
Enrichir les tâches et le quotidien des utilisateurs

Meilleur service
Un service de qualité, plus rapide, pour les partenaires

Aide à la décision
Business intelligence : synthèses et prévisions pour décider

Informatiser = développer des logiciels qui représentent des parties du SI.

Système d’information et système informatique



Système d’information

Toute l’information de l’entreprise, informatisée ou non.

Système informatique

La partie du SI qui peut être automatisée.

Toute entreprise possède un SI. Informatiser, c’est en automatiser une partie.

EN CLAIR

Le SI contient le système informatique

SYSTÈME D'INFORMATION (SI) : TOUTE L'INFORMATION DE L'ENTREPRISE

Pas informatisé

Carnet de commandes papier

Réunion du lundi matin

Note de service affichée

SYSTÈME INFORMATIQUE : LA PARTIE INFORMATISÉE

Logiciel de caisse

Application de commande

Base de données des clients

Une entreprise sans ordinateur a quand même un SI. Informatiser, c'est faire passer une partie du SI dans le système informatique.

Exemples de parties d’un SI

Parties classiques
Comptabilité, paie, facturation

Gestion commerciale
GRC (CRM) : gestion de la relation client

Marketing (SIM)
Collecter et traiter les données pour piloter le marketing

Ressources humaines (GRH)
Carrières, compétences, formations, salaires, congés

Pour automatiser ces parties, il faut concevoir des logiciels avec une méthodologie.

QUIZ · 1 MIN

Lequel de ces éléments relève du système informatique ?

- **A** Une procédure papier de congés
- **B** Une note de service affichée
- **C** La réunion hebdomadaire
- **D** Le logiciel de paie

Réponse D : le logiciel de paie

Le système informatique est la partie informatisée du SI. Les trois autres éléments restent dans le SI sans être informatisés.

À RETENIR · PARTIE 1

Partie 1 : l'essentiel

1. L'entreprise est un système : pilotage, information et opérant, en interaction.
2. Le SI = toute l'information ; le système informatique = sa partie informatisée.
3. Informatiser le SI : productivité, meilleur service, aide à la décision.

DANS L'AGENCE

Opérant : agents au comptoir, garage, voitures. Information : fiches de réservation, contrats, factures. Pilotage : la direction fixe les tarifs et la taille de la flotte.

Question : le cahier papier des réservations fait-il partie du SI ? du système informatique ?

Partie 2

Architecture d’un système d’entreprise

Architecture d’entreprise (ESA) | Architecture logicielle | Conception architecturale

Qu’est-ce que l’ESA ?

Elle se concentre sur la **structure globale** et le **fonctionnement** de l’organisation, en intégrant des éléments stratégiques, métiers et technologiques.

OBJECTIF

Aligner les systèmes informatiques sur les objectifs de l’entreprise, en optimisant les ressources, les processus et les technologies.

TROIS NIVEAUX À ALIGNER

Stratégie
Les objectifs de l’entreprise

Métier
Les processus et les opérations

Technologie
Les systèmes informatiques

Ce qu’apporte une bonne ESA

Analyse de l’architecture	Analyse du système au niveau architectural
Compréhension métier	Comprendre concrètement les opérations
Planification	De la stratégie à l’amélioration locale
Restructuration	Intégrer le système quand l’activité change
Évolution du système	Migration, décommissionnement, nouveaux services

L’architecture logicielle

Composants actifs
Les parties actives du logiciel

+

Mécanisme d’interaction
Comment ils échangent

+

Règles
Elles gouvernent l’interaction

=

Architecture logicielle
Organisation interne du logiciel

Rappel : un logiciel = des programmes (séquences d’instructions) + des données (fichiers ou bases de données).

Structurer un logiciel en composants

1. **Fonctionnalités**
   Définir ce que fait chaque composant

2. **Interfaces**
   Définir comment les composants se parlent

3. **Dépendances**
   Déterminer qui dépend de qui

4. **Sous-composants**
   Décomposer chaque composant en parties plus fines

À vous : quels composants pour une application de réservation de voiture ?

Deux échelles d’architecture

Critère	Architecture d’entreprise	Architecture logicielle
Portée	L’organisation entière	Une application précise
Focus	Structure globale, fonctionnement, éléments stratégiques, métiers et technologiques	Organisation interne : composants et interactions
But	Aligner l’informatique sur les objectifs de l’entreprise	Satisfaire les exigences fonctionnelles et non fonctionnelles



L’ESA fixe le cadre ; chaque application s’y inscrit avec sa propre architecture.

EN CLAIR

La ville et la maison

ARCHITECTURE D'ENTREPRISE

Le plan d'une ville

Quartiers, routes, réseaux d'eau et d'électricité : comment tout fonctionne ensemble.

Pour : direction, responsables métier

ARCHITECTURE LOGICIELLE

Le plan d'une maison

Pièces, portes, escaliers : comment un seul bâtiment est organisé.

Pour : équipes de développement

On ne dessine pas une maison sans connaître son quartier : chaque application s'inscrit dans le cadre de l'entreprise.

La conception architecturale en 4 gestes

Définir la structure
Modules, interfaces (API), dépendances

Choisir des patrons
Couches, microservices, MVC

Exigences non fonctionnelles
Performance, sécurité, scalabilité, maintenabilité, résilience

Choisir les technologies
Langages, bases de données, frameworks, outils

Conception architecturale = proposer une architecture logicielle.

EN CLAIR

Trois façons d’organiser un logiciel

COMME UN IMMEUBLE

Couches

Des étages : écrans, règles métier, données. Chaque étage ne parle qu’à ses voisins.

COMME UN RESTAURANT

MVC

Modèle : la cuisine (données, règles). Vue : la salle (affichage). Contrôleur : le serveur.

COMME UN FOOD COURT

Microservices

Plusieurs petits services indépendants, un seul rôle chacun. On en change un sans toucher aux autres.

Un patron est une organisation éprouvée : on ne réinvente pas le plan à chaque projet.

Patron 1 : les couches



RÈGLE

Chaque couche ne parle qu'à celle du dessus et à celle du dessous.

AVANTAGE

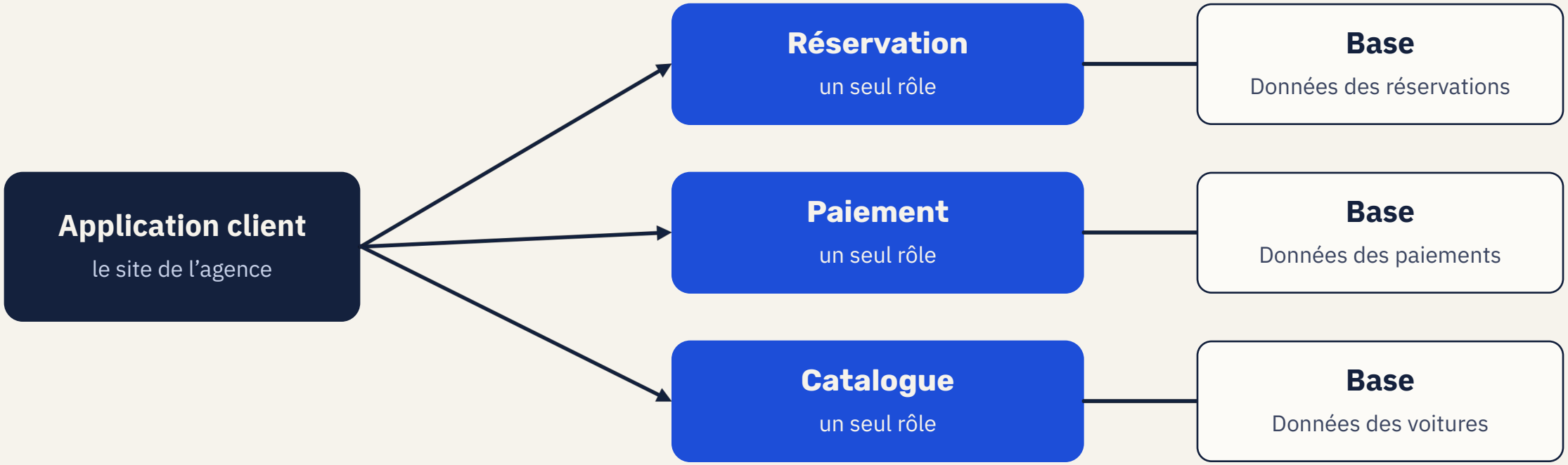
On peut refaire les écrans sans toucher à la base de données.

Patron 2 : MVC



Le contrôleur reçoit, le modèle garde les voitures et les règles, la vue montre le résultat.

Patron 3 : les microservices



Un rôle par service : on en modifie un sans toucher aux autres.

QUIZ · 1 MIN

Lequel de ces éléments est un patron d'architecture ?

- **A** MySQL
- **B** MVC
- **C** Figma
- **D** Scrum

Réponse B : MVC

C'est un patron d'architecture, comme les couches ou les microservices. MySQL est une technologie, Figma un outil, Scrum une méthode agile.

À RETENIR · PARTIE 2

Partie 2 : l'essentiel

1. L'architecture d'entreprise aligne l'informatique sur les objectifs de l'entreprise.
2. L'architecture logicielle organise un logiciel : composants, interactions, règles.
3. Concevoir : découper, choisir un patron et des technologies, viser la qualité.

DANS L'AGENCE

Ville : l'agence a plusieurs applications (réservation, facturation, flotte) qui doivent s'entendre. Maison : le logiciel de réservation, organisé en couches ou en MVC.

Question : quels composants pour le logiciel de réservation de l'agence ?

Partie 3

Mise en œuvre d’un système d’entreprise

Cycle de vie et approches | Méthodologies classiques | Processus unifiés | Méthodes agiles

Le cycle de vie d’un logiciel

Modélisation conventionnelle de la succession d’étapes de mise en œuvre d’un produit logiciel, **de la perception du besoin à la livraison**.

ÉTAPE 1
Étude préalable
Comprendre l’existant

→

ÉTAPE 2
Spécification
Formaliser les besoins

→

ÉTAPE 3
Analyse
Unifier les cas d’utilisation

ÉTAPE 4
Conception
Choisir l’architecture

→

ÉTAPE 5
Implémentation
Produire les composants

→

ÉTAPE 6
Test
Valider le logiciel

Trois approches de mise en œuvre d'un SI

DAG

Développement d'Application de Gestion

Objectif : automatiser, contrôler les coûts

Destinataire : la machine

Données : sémantique pauvre

SAO

Système d'Application Organisationnel

Objectif : aider à décider

Destinataire : l'humain

Données : sémantique riche

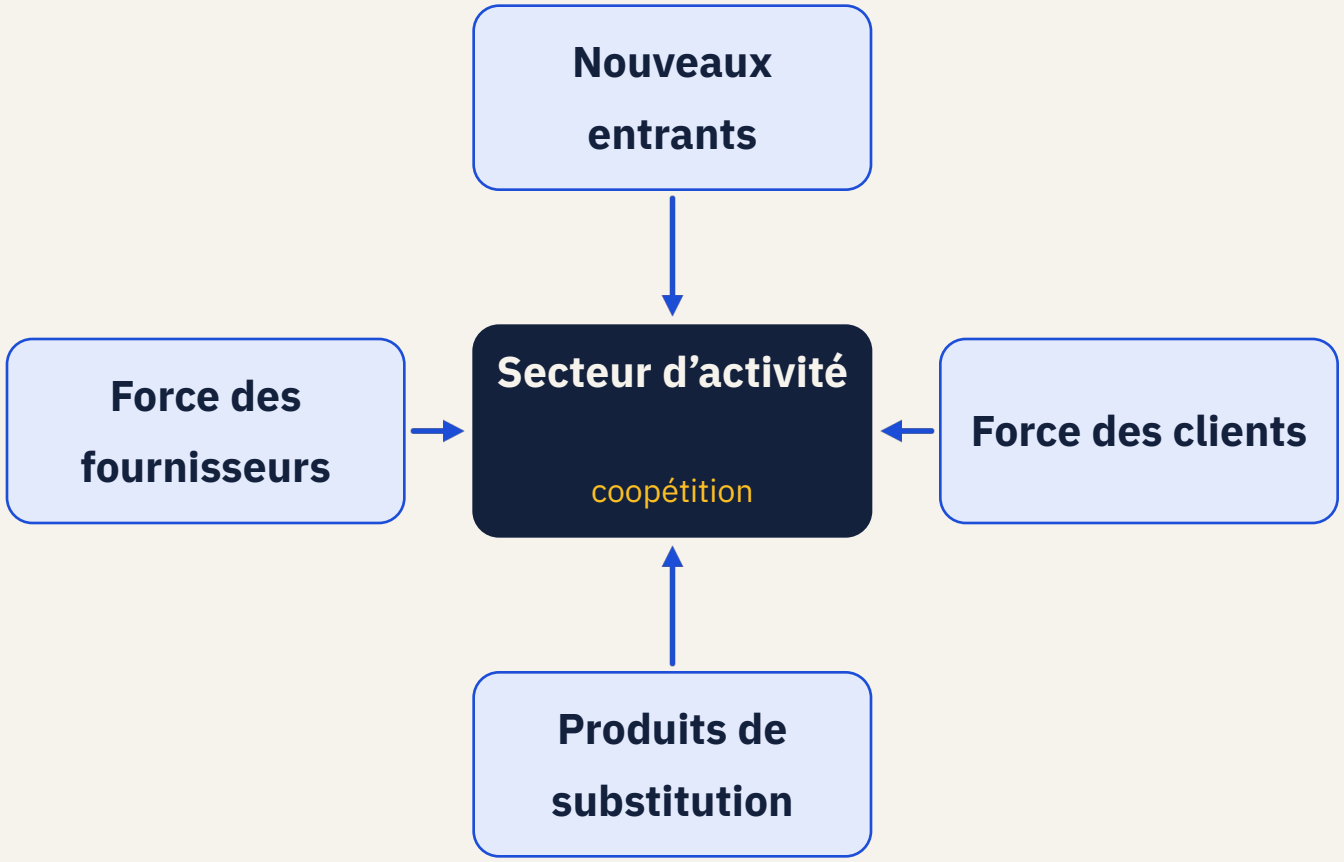
SIS

Système d'Information Stratégique

Objectif : avantage concurrentiel

Vision : le SI différencie l'offre

Le SI, un avantage concurrentiel



FACTEURS CLÉS D'UN SIS

- Avantage pour le client
- Différenciation de l'offre
- Pouvoir de négociation
- Barrières d'entrée
- Catalyseur d'innovation

Le SI doit aider à atteindre une position favorable dans le nouvel équilibre des forces.

QUIZ · 1 MIN

Un SI riche en sémantique pour aider les humains à décider : quelle approche ?

- **A** SAO : application organisationnelle
- **B** DAG : application de gestion
- **C** SIS : information stratégique
- **D** Aucune des trois

Réponse A : SAO

Il représente et mesure les activités de l'organisation, avec des informations riches destinées à l'humain pour l'aide à la décision.

Une méthode d’informatisation

Elle définit un processus d’informatisation, avec une **portée** (le domaine étudié) et une **démarche** (des travaux ordonnés). Elle repose sur quatre composants :

COMPOSANT 1

Modèles

Concepts et règles pour représenter l’organisation

COMPOSANT 2

Langages

Pour écrire les spécifications et communiquer

COMPOSANT 3

Démarche

Un ensemble de travaux ordonnés par étapes

COMPOSANT 4

Outils et techniques

Pour manipuler modèles, langages et démarche



« Une méthode ne remplace ni l’expérience, ni la connaissance, ni le talent. »

Taxinomie des méthodes : 4 fondements

Cartésienne

Démarche analytique et descendante : du général au particulier

SADT · CORIG

Systémique

Démarche globalisante : modélisation par niveaux d'abstraction

MERISE · AXIAL · IA-NIAM

Objet

Le paradigme objet appliqué à tout le processus

OOA · OMT · HOOD · OOSE · Merise Objet

Formelle

Spécifications mathématiques, ensuite prouvées

B · Z

Un SI robuste : 3 facteurs clés

1

Une vision claire

Une présentation stratifiée du système

2

Un support technologique adéquat

Des choix techniques adaptés aux besoins

3

L'adhésion

Celle des acteurs concernés par le système

À débattre : lequel de ces trois facteurs est le plus souvent négligé ?

Projet, démarche et méthode

Projet	Atteindre un objectif. Quelle est la meilleure manière d'y arriver ?
Démarche (méthodologie)	Principes généraux qui guident le choix d'une méthode adaptée à un projet.
Méthode	Description pas-à-pas des étapes, avec des outils. Spécifique à un projet.

✓ **La méthode permet de statuer sur l'état d'avancement du projet.**

Trois familles de méthodologies

Classiques (SDLC)

Planifiées, séquentielles, rigides

- Modèle en cascade
- Modèle en V
- Modèle en spirale
- Modèle incrémental

Processus unifiés

Itératifs, guidés par les cas d’utilisation

- Two Tracks UP
- eXtreme UP
- Enterprise UP
- Rational UP

Méthodes agiles

Itératives, collaboratives, adaptatives

- Scrum
- Kanban
- RAD
- XP

REPÈRES · APPROXIMATIFS

Des classiques à l'agile

Années 1970

Cascade

Chaque étape finie avant la suivante.

Années 1980

V et spirale

Tests en face de chaque phase ; risques analysés à chaque tour.

Années 1990

UP, RAD, XP, Scrum

Itérations, prototypes, livraisons fréquentes.

Année 2001

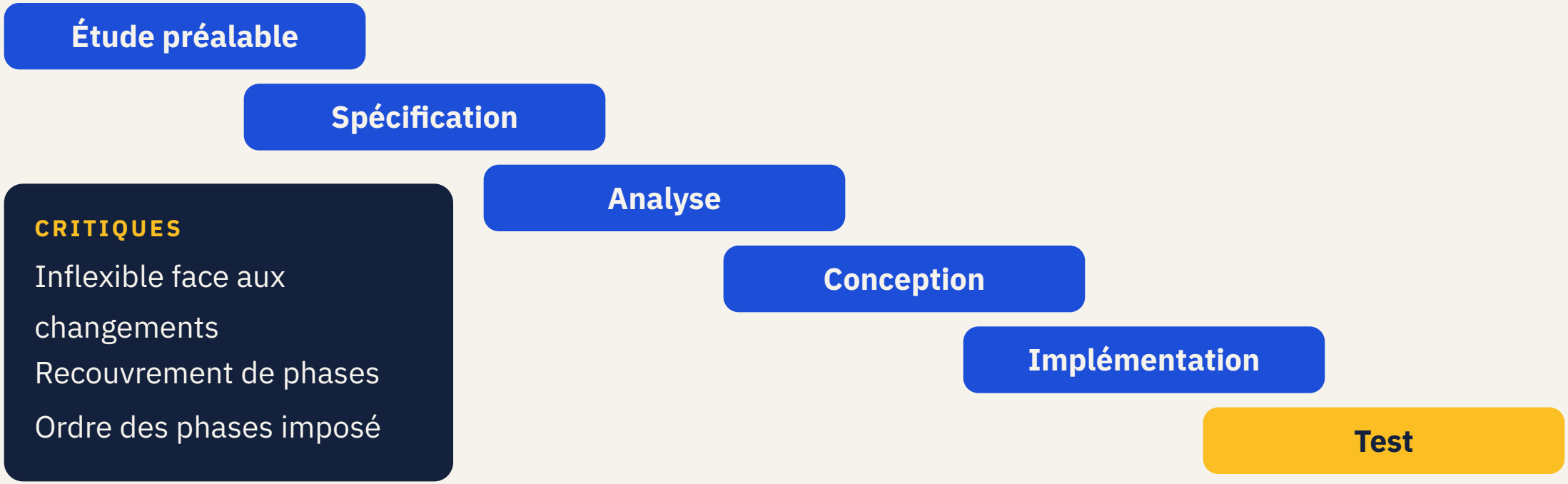
Manifeste agile

Quatre valeurs : on s'adapte au changement.

Les classiques planifient tout ; les agiles s'adaptent en livrant souvent.

Le modèle en cascade (Waterfall)

Une séquence linéaire : chaque étape est terminée avant de passer à la suivante.



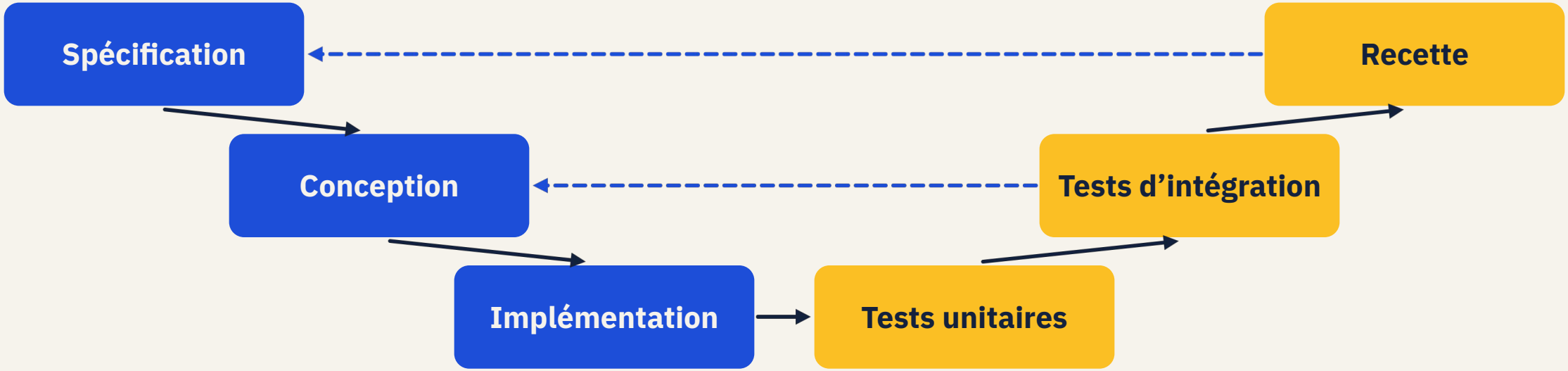
CRITIQUES

Inflexible face aux changements

Recouvrement de phases

Ordre des phases imposé

Le modèle en V



Chaque phase de développement a sa phase de test correspondante.

Si une phase échoue aux tests, retour en arrière pour la corriger.

Le modèle en spirale

1 Objectifs

Objectifs du cycle, alternatives et contraintes

2 Risques

Analyse des risques, évaluation des alternatives, maquettage

4 Vérification

Vérification des résultats et planification du cycle suivant

3 Développement

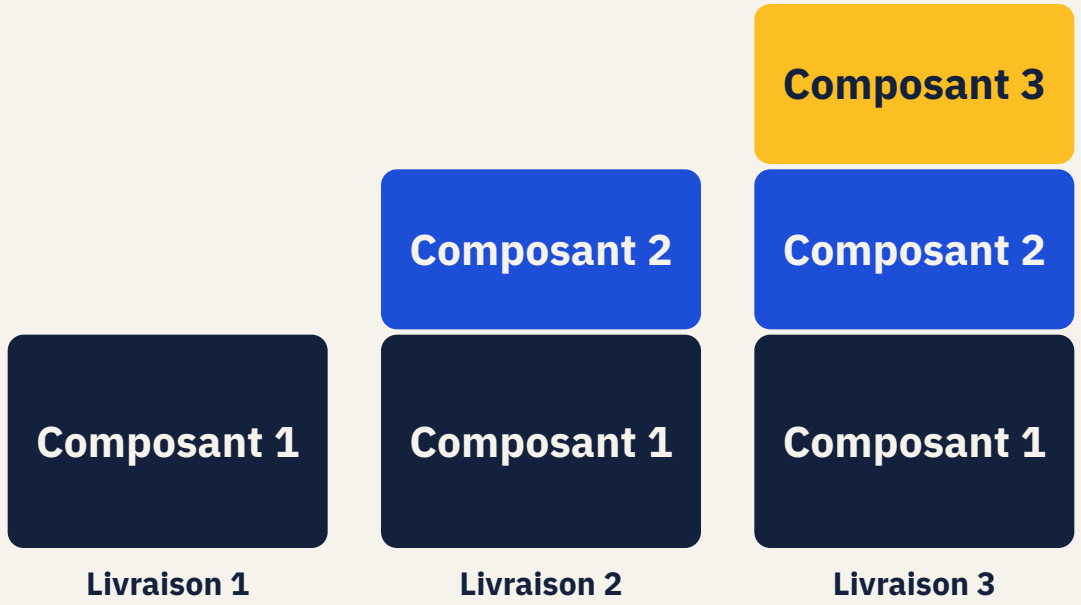
Conception et développement (cascade ou V)

À RETENIR

- Chaque tour livre une version du logiciel
- L’analyse des risques est centrale
- Adapté aux systèmes innovants de grande taille
- Combinable avec d’autres méthodes

Le modèle incrémental

- Des sous-ensembles fonctionnels, livrés l'un après l'autre
- Le premier composant coûte le plus : il pose toute l'infrastructure
- Chaque composant suit son propre cycle, puis s'intègre à la solution déjà livrée



EN CLAIR

Quatre façons de construire une maison

Cascade

Chaque étape est finie avant la suivante : plans, fondations, murs, toit.

Besoins stables

Modèle en V

Comme la cascade, avec une inspection qui correspond à chaque étape.

Exigences figées, tests critiques

Spirale

On teste sur maquette ce qui est risqué, on construit, puis on recommence.

Projet innovant et risqué

Incrémental

On livre une pièce habitable à la fois : cuisine, puis chambre, puis salon.

Livraisons progressives

Le processus unifié (UP) : 3 principes

Démarche mise en œuvre par les auteurs d'UML : Booch, Jacobson et Rumbaugh.

Guidé par les cas d'utilisation

Capturer les besoins fonctionnels et orienter chaque itération

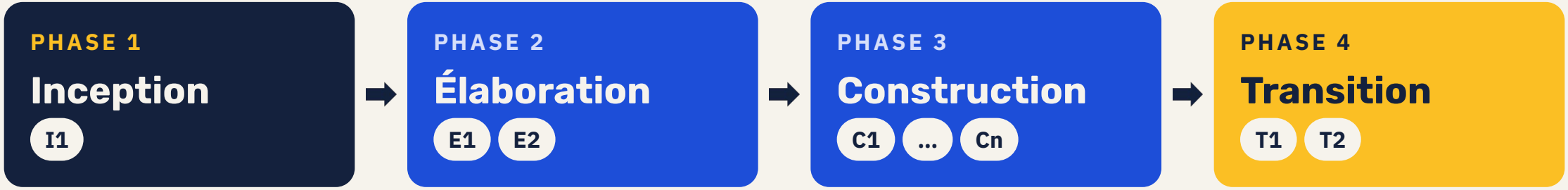
Centré sur l'architecture

Définir l'architecture type du système dès que possible

Itératif et incrémental

Chaque itération produit un livrable, par incréments successifs

Les 4 phases du processus unifié



Chaque itération produit un incrément

Planification → Besoins, analyse, conception, implémentation, test → Évaluation → Prototype exécutable

Quelle activité à quelle phase ?

ACTIVITÉ	Inception	Élaboration	Construction	Transition
Besoins	Fort	Fort	Faible	Faible
Analyse	Moyen	Fort	Moyen	Faible
Conception	Faible	Fort	Fort	Faible
Implémentation	Faible	Moyen	Fort	Moyen
Test	Faible	Moyen	Fort	Fort

Intensité relative d'après le processus unifié (RUP) : illustration, pas dans le support de cours.

EN CLAIR

Les 4 phases en un mot

PHASE 1

Inception

Démarrer : cadrer le projet et son périmètre.

PHASE 2

Élaboration

Préparer : plan solide, architecture, risques principaux.

PHASE 3

Construction

Fabriquer : développer le produit, itération après itération.

PHASE 4

Transition

Livrer : mettre le produit entre les mains des utilisateurs.



Une phase contient plusieurs itérations ; chaque itération ajoute un incrément au produit.

Rational Unified Process (RUP)

Guides et pratiques
Modèles et bonnes pratiques pour chaque phase

Personnalisation
Le processus s'adapte aux besoins du projet

Gestion du risque
Changements et risques gérés à chaque itération

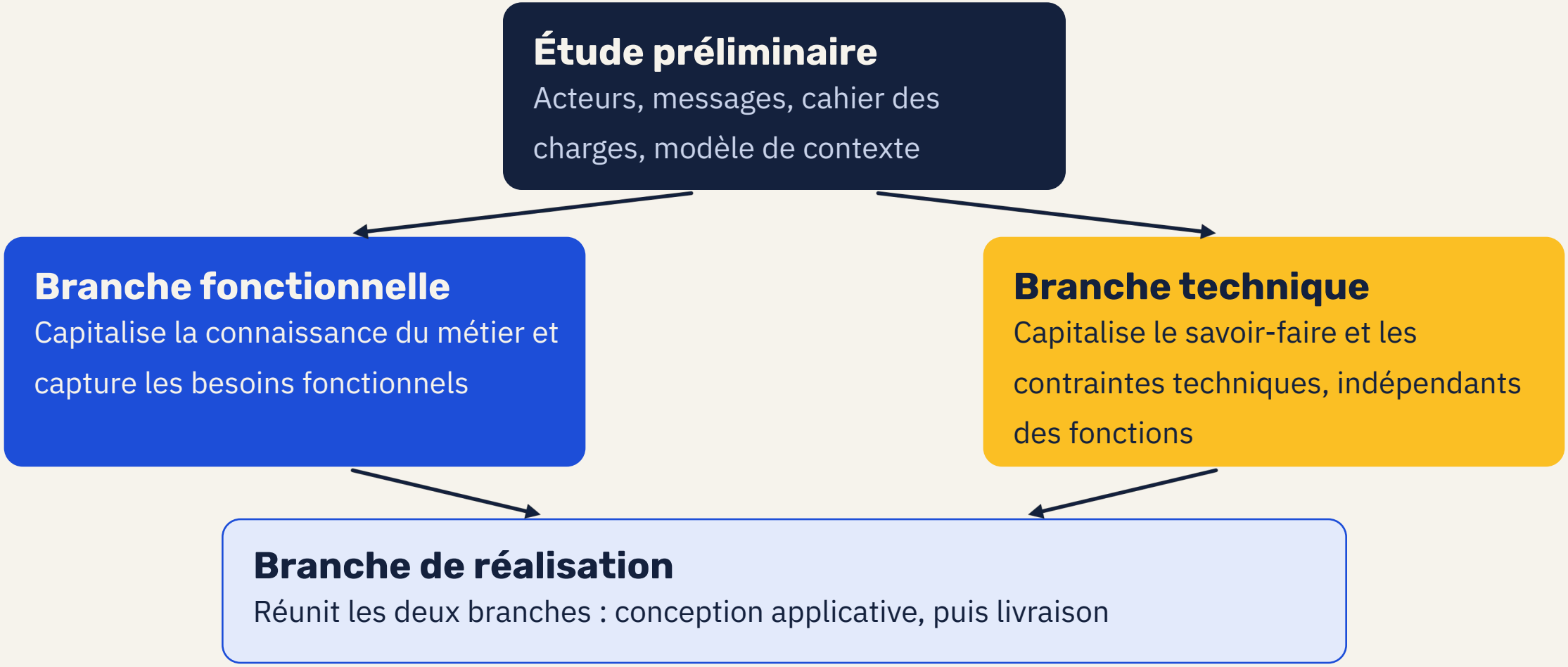
Cas d'utilisation
Directives détaillées pour capturer les exigences

Architecture
Une architecture solide, documentée dès le début

Ouverture à l'agile
Itérations et collaboration étroite avec le client

☆ **RUP = le processus unifié, avec des guides détaillés et plus de personnalisation.**

Two Tracks UP (2TUP) : un cycle en Y



QUIZ · 1 MIN

Quelles sont les 4 phases du processus unifié, dans l'ordre ?

A Analyse, conception, codage, test

B Spécification, développement, recette, maintenance

C Inception, élaboration, construction, transition

D Planification, itération, évaluation, livraison

Réponse C : inception, élaboration, construction, transition

Chaque phase compte plusieurs itérations. L'option D décrit plutôt le contenu d'une itération.

Le manifeste Agile : 4 valeurs

Personnes et interactions	*plus que*	Processus et outils
Un produit opérationnel	*plus que*	Une documentation exhaustive
Collaboration avec le client	*plus que*	Négociation d’un contrat
Adaptation au changement	*plus que*	Suivi d’un plan

La documentation reste un support complémentaire, pas l’objectif.

Les points forts des méthodes agiles

Échanges constants avec le client

Documentation réduite, productivité en hausse

Collaboration quotidienne avec le client

Version fonctionnelle livrée fréquemment

Excellence technique : tests en continu

Progrès visible, pas de mauvaises surprises

Gain de productivité et baisse des coûts de production.

Scrum : le cycle d’un sprint

SPRINT · 2 SEMAINES À 1 MOIS, IDÉALEMENT 2 SEMAINES

ÉTAPE 1

Backlog du produit

Besoins du client, classés par priorité

→

ÉTAPE 2

Backlog du sprint

Éléments retenus pour ce sprint

→

ÉTAPE 3

Sprint

Développement, avec une mêlée chaque jour

→

ÉTAPE 4

Incrément

Ajouts au produit, présentés en fin de sprint

Fin du sprint : on livre l’incrément, puis le cycle recommence.

Scrum : 3 rôles

Product Owner

Voix du client

Transmet les objectifs du client à l'équipe et coordonne l'implication des utilisateurs.

Scrum Master

Facilitateur

Optimise la capacité de l'équipe et favorise son autonomie.

L'équipe

Idéalement moins de 10

S'auto-organise : pas de hiérarchie interne, tous réalisent le produit.

Le Scrum Master facilite le travail de l'équipe ; il ne la commande pas.

Le vocabulaire de Scrum

Backlog du produit

Exigences hiérarchisées avec le client ; la liste évolue tout au long du projet.

Backlog du sprint

Éléments du backlog produit que l'équipe choisit de réaliser pendant le sprint.

User story

Une fonctionnalité décrite par le client, du point de vue de l'utilisateur.
« En tant que client, je veux réserver une voiture en ligne afin de gagner du temps. »

Mêlée

Réunion d'avancement quotidienne pendant le sprint : où en est-on, qu'est-ce qui bloque ?

QUIZ · 1 MIN

Qui aide l'équipe à travailler de façon autonome tout en s'améliorant ?

- **A** Le client
- **B** Le Product Owner
- **C** Le chef de projet
- **D** Le Scrum Master

Réponse D : le Scrum Master

Il optimise la capacité de l'équipe. Le Product Owner, lui, porte la voix du client.

RAD : Rapid Application Development

1. Le prototypage

Ignore l'analyse et la conception : on implémente directement un prototype du système réel, avec de fausses données.
Pour voir au plus vite si le résultat correspond à ce qui est souhaité.

2. Développement par l'utilisateur final

Les utilisateurs finaux créent leurs propres applications.
Ils sont formés pour développer des applications personnalisées.

But du RAD : livrer rapidement un minimum de fonctions opérationnelles.

eXtreme Programming (XP)

Une méthode agile pour les **petites équipes** qui travaillent dans un **contexte changeant**.

4 PRINCIPES CLÉS

- **Simplicité** : le nécessaire, sans fonctionnalité superflue
- **Feedback rapide** : retours constants du client, tests fréquents
- **Amélioration continue** : refactorer le code régulièrement
- **Collaboration** : forte communication entre développeurs et avec le client

4 PRATIQUES PRINCIPALES

- **Développement itératif** : petites itérations de 1 à 2 semaines
- **TDD** : les tests automatisés sont écrits avant le code
- **Pair programming** : en binôme, pour écrire et relire le code
- **Intégration continue** : le code est fréquemment intégré et testé

Classique ou agile : ce qui change

Critère	Méthodes classiques	Méthodes agiles
Approche	Planifiée, séquentielle, rigide	Itérative et collaborative
Changement	Difficile en cours de route	Intégré au développement
Documentation	Exhaustive	Réduite
Client	Contrat négocié au départ	Collaboration quotidienne
Livraison	Souvent en fin de projet	Version fonctionnelle fréquente
Exemples	Cascade, V, spirale, incrémental	Scrum, Kanban, RAD, XP

VOTE · LEVEZ LA MAIN

Quelle méthode pour chaque projet ?

1. Dispositif médical : exigences figées, chaque étape doit être validée par des tests.

   ✓ **Modèle en V**
   Chaque phase a son test

2. Application mobile d'une startup : les besoins changent chaque semaine.

   ✓ **Agile / Scrum**
   Le changement est intégré

3. Refonte du SI d'une grande banque : projet innovant, risque d'échec élevé.

   ✓ **Modèle en spirale**
   Analyse des risques

GUIDE

Choisir une méthode en trois questions



Un guide, pas une règle : plusieurs réponses se défendent.

À RETENIR · PARTIE 3

Partie 3 : l’essentiel

1. Trois familles : classiques (cascade, V, spirale, incrémental), processus unifiés, agiles.
2. Classique : on planifie tout d’abord. Agile : on livre souvent et on s’adapte.
3. Le choix dépend de la stabilité des besoins et du niveau de risque.

DANS L’AGENCE

Paiement et contrat à valider à chaque étape : modèle en V. Application mobile qui évolue chaque semaine : Scrum. Plusieurs choix se défendent.

Question : quelle méthode pour l’appli mobile de l’agence ? Justifiez avec les critères du cours.

Partie 4

Les activités de développement logiciel

Étude préalable | Besoins | Analyse | Conception | Implémentation | Tests

Les 6 étapes de création d'un logiciel

1 Étude préalable

Diagnostic de l'existant, cahier des charges

→ Modèle des cas d'utilisation

2 Spécification des besoins

Besoins fonctionnels, non fonctionnels et techniques

→ Modèle des cas d'utilisation

3 Analyse

Vision unifiée des cas d'utilisation, premières classes

→ Modèle d'analyse

4 Conception

Architecture, base de données, vues UML, maquettes

→ Conception + déploiement

5 Implémentation

Mise en œuvre : produire les composants logiciels

→ Modèle d'implémentation

6 Test

Vérifier que le logiciel répond aux besoins

→ Modèle des tests

EN CLAIR

Une question par étape

1 · ÉTUDE PRÉALABLE

Faut-il le faire, et est-ce faisable ?

2 · BESOINS

Que veut le client ?

3 · ANALYSE

Quelles sont les grandes pièces du système ?

4 · CONCEPTION

Comment le construire, avec quelle technologie ?

5 · IMPLÉMENTATION

Comment l'écrire en code ?

6 · TEST

Est-ce que ça marche comme prévu ?

Comme pour une maison : du terrain à l'inspection finale.

Étude préalable : 4 objectifs

1 Diagnostic

Analyser le fonctionnement de l'entreprise et faire un diagnostic général de l'existant.

2 Critiques et besoins

Recenser les critiques, positives ou négatives, sur l'entreprise et son système informatique, ainsi que les besoins des utilisateurs.

3 Opportunité et faisabilité

Évaluer l'opportunité (financement, moyens humains…) et la faisabilité technique des automatisations.

4 Cahier des charges

Le rédiger : c'est le livrable de l'étude préalable.

Étude préalable : 3 étapes

ÉTAPE 1

Définir le contexte

Cadre du projet, activités de l'entreprise, objectifs du nouveau système

ÉTAPE 2

Étudier l'existant

Décrire les solutions existantes et corriger leurs dysfonctionnements

ÉTAPE 3

Choisir le cycle de développement

Comparer les méthodologies, retenir la plus adaptée aux besoins

MODÉLISATION MÉTIER

- Diagramme de contexte métier
- BPMN
- Diagramme d'activité
- Cas d'utilisation métier

Le choix de la méthode vue en partie 3 se fait ici, dès l'étude préalable.

Trois familles de besoins

Besoins fonctionnels

QUOI FAIRE ?

- Schéma conceptuel des traitements et des données
- Indépendant de tout matériel ou logiciel
- Complet, cohérent, réalisable

Besoins non fonctionnels

QUALITÉS ATTENDUES

- Performance
- Sécurité
- Ergonomie et convivialité
- Maintenance et évolution

Besoins techniques

CHOIX TECHNIQUES

- Structures de données
- Unités de traitement
- Choix des matériels
- Calendrier et budgets

Fonctionnel : ce que fait le système. Non fonctionnel : comment il le fait.

Capturer puis analyser les besoins

Capture des besoins

1. **Besoins fonctionnels** : acteurs, diagramme de contexte, expression littéraire
2. **Besoins non fonctionnels** : performance, sécurité, ergonomie…
3. **Spécification** : diagrammes de cas d’utilisation (généraux, détaillés)

Analyse des besoins

1. Description textuelle des cas d’utilisation
2. Diagrammes de séquence d’analyse
3. Diagrammes d’état-transition
4. Diagrammes d’activité

Capturer, c’est écouter le client ; analyser, c’est préparer les développeurs.

QUIZ · 1 MIN

Laquelle de ces exigences est non fonctionnelle ?

- **A** Le système répond en moins de 2 secondes
- **B** Le client réserve une voiture en ligne
- **C** Le client consulte la liste des véhicules
- **D** Le système enregistre la demande

Réponse A : une exigence de performance

Elle porte sur la qualité du service, pas sur une fonction. Les trois autres disent ce que fait le système.

Recueil des besoins ou analyse ?

Critère	Recueil des besoins	Modèle d’analyse
Langage	Celui du client	Celui du développeur
Vue	Externe du système	Interne du système
Structure	Cas d’utilisation	Packages et stéréotypes
Rôle	Contrat entre clients et développeurs	Guide les développeurs pour la conception

L’analyse unifie les cas d’utilisation et fait apparaître un premier jeu complet de classes.

La conception : deux volets

Conception architecturale

CHOIX DU MODÈLE ARCHITECTURAL

- 1.1 Architecture physique
- 1.2 Architecture logique (motif d'architecture logicielle)

Conception détaillée

TROIS ACTIVITÉS

2. Base de données : MCD, puis MLD
3. Conception logicielle : vue statique et vue dynamique
4. Conception graphique : maquettage

D'abord la structure globale, ensuite le détail de chaque partie.

Architecture logicielle ou matérielle ?

</> Architecture logicielle

- Modules, composants, couches
- Responsabilités séparées, dépendances maîtrisées

- Motifs : MVC, microservices, couches
- Interfaces entre les parties

Architecture matérielle (physique)

- Serveurs, machines virtuelles
- Dispositifs embarqués, autres infrastructures
- Influence la performance, la disponibilité, l’évolutivité

La conception architecturale vise surtout le logiciel, parfois aussi le matériel.

Concevoir l’architecture, c’est…

1. **Estimer les besoins physiques** : performance, concurrence, gestion des données, IHM, robustesse
2. **Organiser le système en sous-systèmes**
3. **Choisir un style architectural**
4. **Intégrer l’existant**
5. **Choisir un modèle de déploiement**

Analyse ou conception ?

Critère	Modèle d'analyse	Modèle de conception
Nature	Modèle conceptuel	Modèle physique
Portée	Autorise plusieurs conceptions	Spécifique à une implantation
Stéréotypes	Peu de stéréotypes	Dépendent de l'environnement cible
Couches	Peu de couches	Plusieurs couches

On passe du conceptuel au physique : une analyse, plusieurs conceptions possibles.

Conception logicielle : deux vues UML

Vue statique

DIAGRAMMES DE STRUCTURE

- Classes
- Objets
- Composants
- Déploiement
- Paquetage
- Structure composite
- Profils

Vue dynamique

COMPORTEMENT

- États-transitions
- Activité

INTERACTION

- Séquence
- Communication
- Global d'interaction
- Temps

Statique : de quoi le système est fait. Dynamique : comment il se comporte.

EN CLAIR

Un diagramme, une question

Diagramme	La question à laquelle il répond
Cas d’utilisation	Qui peut faire quoi avec le système ?
Classes	De quoi le système est-il fait ?
Séquence	Dans quel ordre les éléments s’envoient-ils des messages ?
États-transitions	Quels états un objet traverse-t-il ?
Activités	Quel est l’enchaînement des actions ?
Composants	Quels blocs logiciels, et comment sont-ils reliés ?
Déploiement	Sur quelles machines chaque bloc est-il installé ?

Base de données : du MCD au MLD

ÉTAPE 1

MCD : modèle conceptuel

Représenter fidèlement le domaine : entités, associations, attributs, cardinalités

→

ÉTAPE 2

MLD : modèle logique

Traduire pour un SGBD précis : tables, colonnes, clés primaires et étrangères

→

RÉSULTAT

Base de données

Stocker, gérer et accéder aux données de façon efficace, sécurisée, cohérente

Le MCD est indépendant du SGBD ; le MLD est spécifique à un SGBD.

Du maquettage à l'implémentation

Conception graphique

Conception visuelle : sites web, applications mobiles, interfaces utilisateur...

OUTILS DE MAQUETTAGE

- Balsamiq
- Figma
- Adobe XD
- Sketch

Implémentation

Produire les composants logiciels à partir des modèles précédents.

1. Environnement de développement : matériel et logiciel
2. Framework front-end
3. Framework back-end
4. Réalisation : diagramme de navigation, interfaces

Merise ou UML ?

Critère	Merise	UML
Nature	Méthode d'analyse et de conception de SI	Langage de représentation d'un SI
Orientation	Données et traitements, bases relationnelles	Notation orientée objet
Paradigme	Relationnel	Objet
Portée	Franco-français	Internationale
Contenu	Schéma directeur, étude préalable, étude détaillée, réalisation	Diagrammes : statique, dynamique…

Merise structure la démarche ; UML fournit le langage de modélisation.

QUIZ · 1 MIN

Dans quelle étape rédige-t-on le cahier des charges ?

- **A** Analyse
- **B** Conception
- **C** Étude préalable
- **D** Implémentation

Réponse C : l'étude préalable

On y diagnostique l'existant, on évalue la faisabilité, puis on rédige le cahier des charges.

À RETENIR · PARTIE 4

Partie 4 : l'essentiel

1. Six étapes : étude préalable, besoins, analyse, conception, implémentation, test.
2. Chaque étape produit des modèles, surtout UML ; les données vont du MCD au MLD.
3. L'analyse dit quoi, sans technologie ; la conception dit comment, avec une technologie.

DANS L'AGENCE

Étude préalable : cahier des charges. Besoins : « Réserver une voiture ». Analyse : classes Client, Réservation, Voiture. Conception : base MySQL, écrans web.

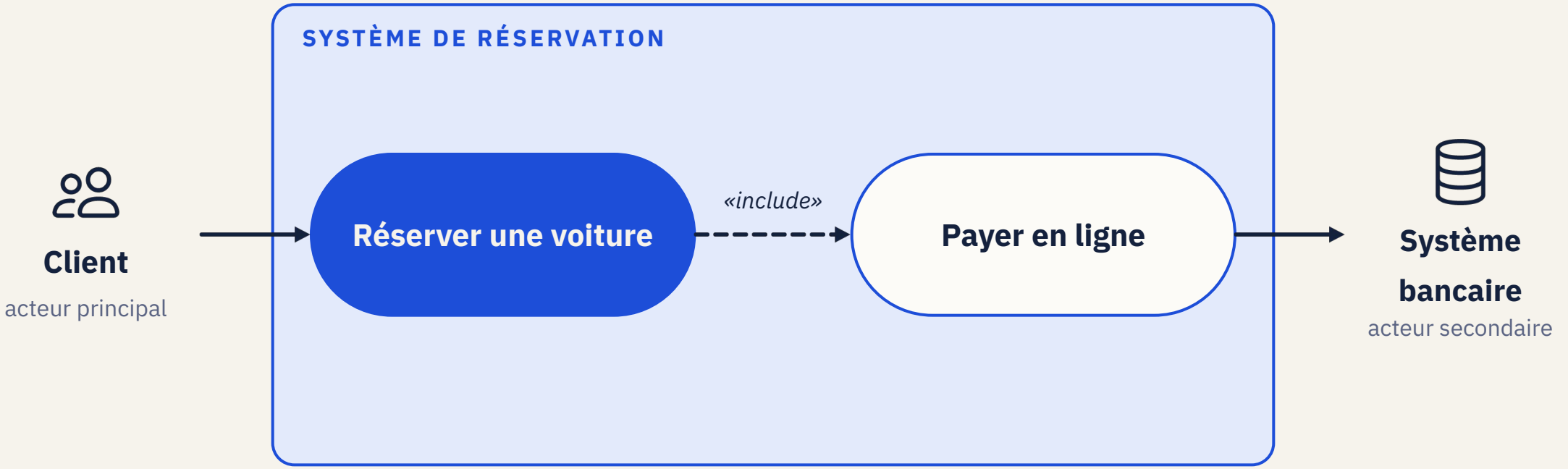
Question : à quelle étape choisit-on d'utiliser MySQL pour la base de l'agence ?

Annexes

UML, MCD et MLD

Cas d’utilisation | Diagrammes UML | MCD | MLD

Cas d’utilisation : réserver une voiture



Un cas d’utilisation est un objectif du système, motivé par le besoin d’un acteur.

Décrire un cas d’utilisation

Titre	Réserver une voiture
But	Permettre à un client internaute de saisir une demande de réservation
Acteurs	Principal : Client · Secondaire : Système bancaire
Pré-condition	Le parc de véhicules n’est pas vide
Post-condition	Une demande de réservation est enregistrée avec toutes les informations nécessaires

📖 **Puis les scénarios : nominal, alternatifs (variantes) et d’exception (erreurs).**

UML, axe statique : quatre diagrammes

Diagramme de classes

Montre la structure interne du système : une description purement statique.

Diagramme d’objets

Illustre des structures de classes compliquées et en vérifie l’adéquation.

Diagramme de composants

Montre le découpage du système en unités logicielles pouvant être distribuées.

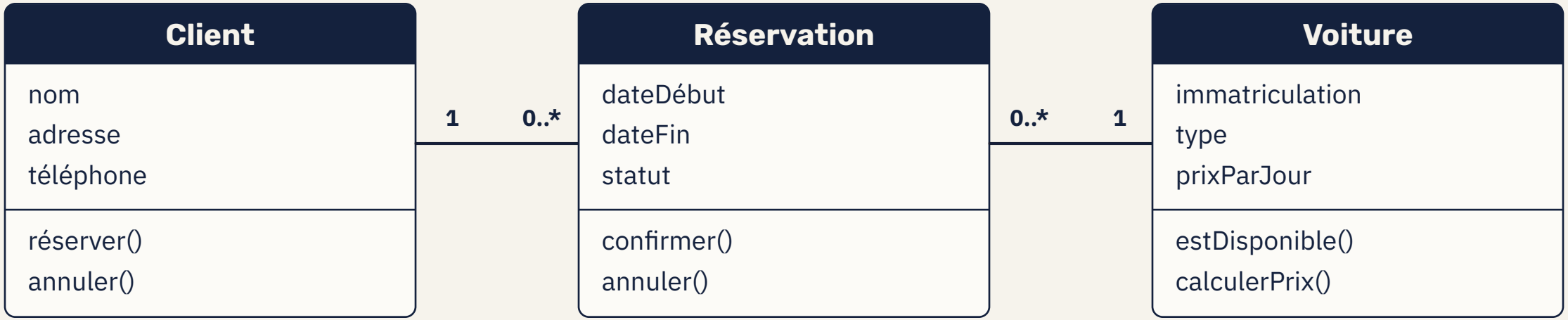
Diagramme de déploiement

Représente le système informatique qui porte le logiciel et l’installation des composants.

Cas d’utilisation : le système vu des acteurs. Classes : sa structure interne.

SCHÉMA

Diagramme de classes : l'agence



Un client a 0 à n réservations ; une réservation concerne un client et une voiture. En UML, le nombre se lit à l'autre bout du trait : c'est l'inverse du MCD.

UML, axe dynamique : quatre diagrammes

Diagramme de séquence

Remplace la boîte noire du système par une collaboration d'objets échangeant des messages.

Diagramme de communication

Montre les relations structurelles entre les objets qui communiquent.

Diagramme d'états

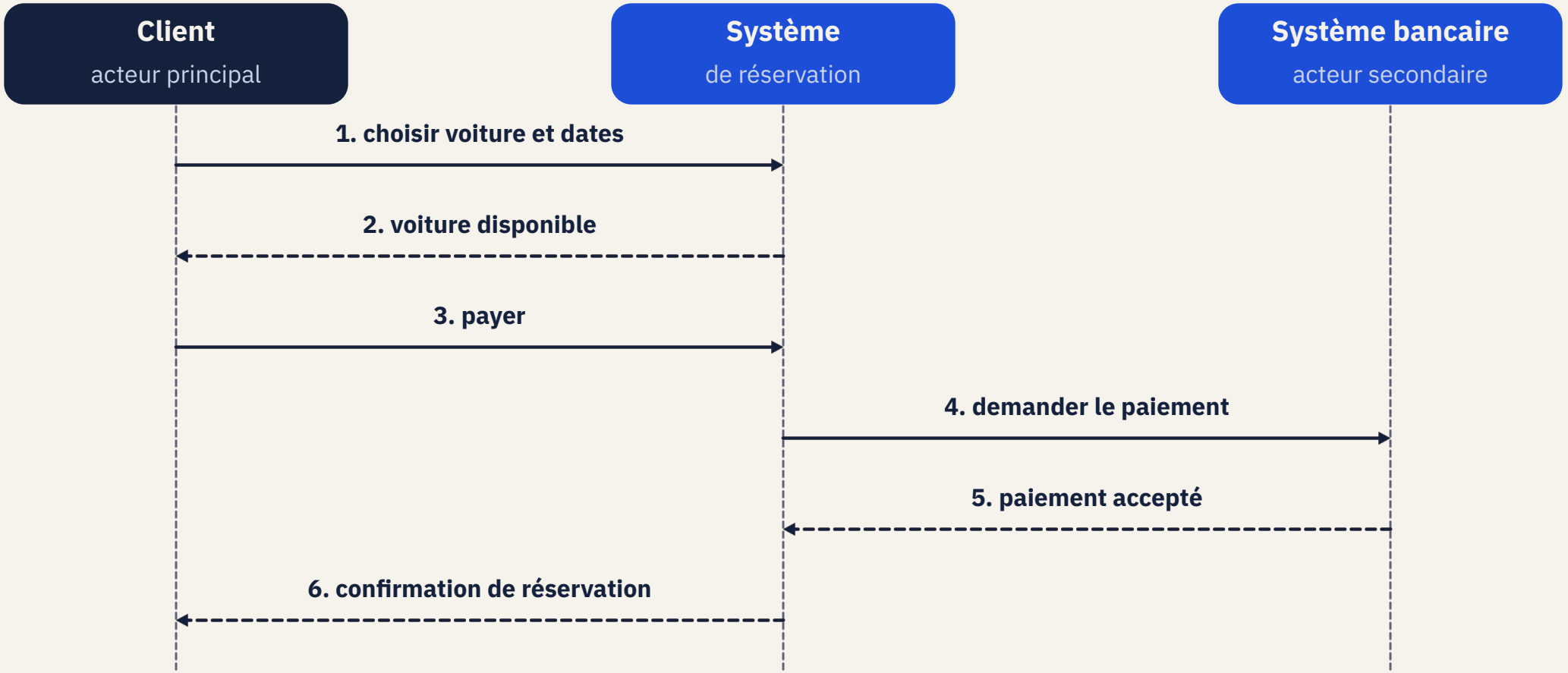
Modélise les effets des interactions sur la configuration interne des objets.

Diagramme d'activités

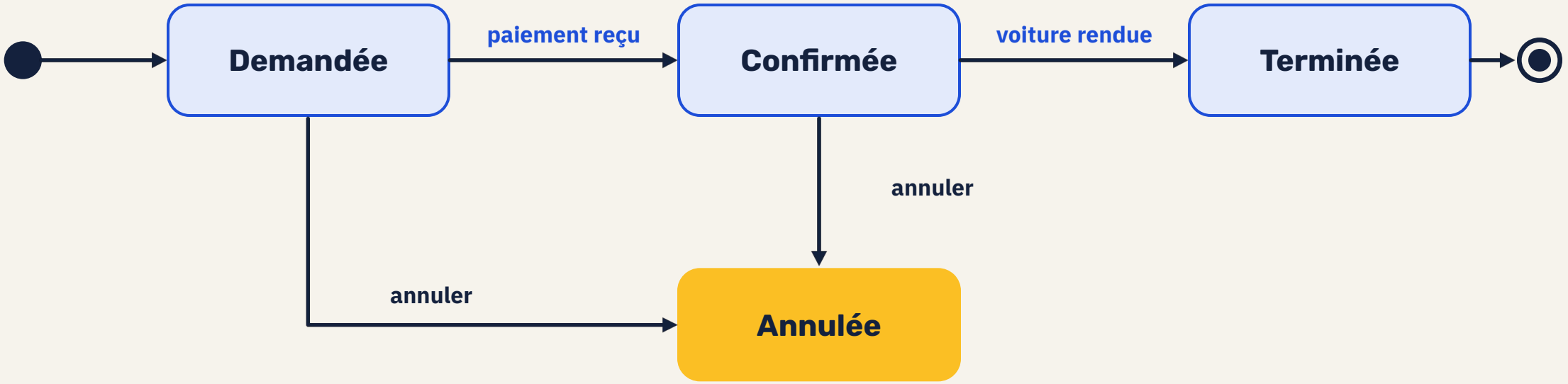
Enchaîne les activités : point de départ, décisions, point d'arrivée.

Les diagrammes dynamiques font le pont entre vision externe et vision interne.

Séquence : réserver une voiture



États : la vie d’une réservation



Un état est une situation de la réservation ; une transition est l’événement qui la fait changer.

QUIZ · 1 MIN

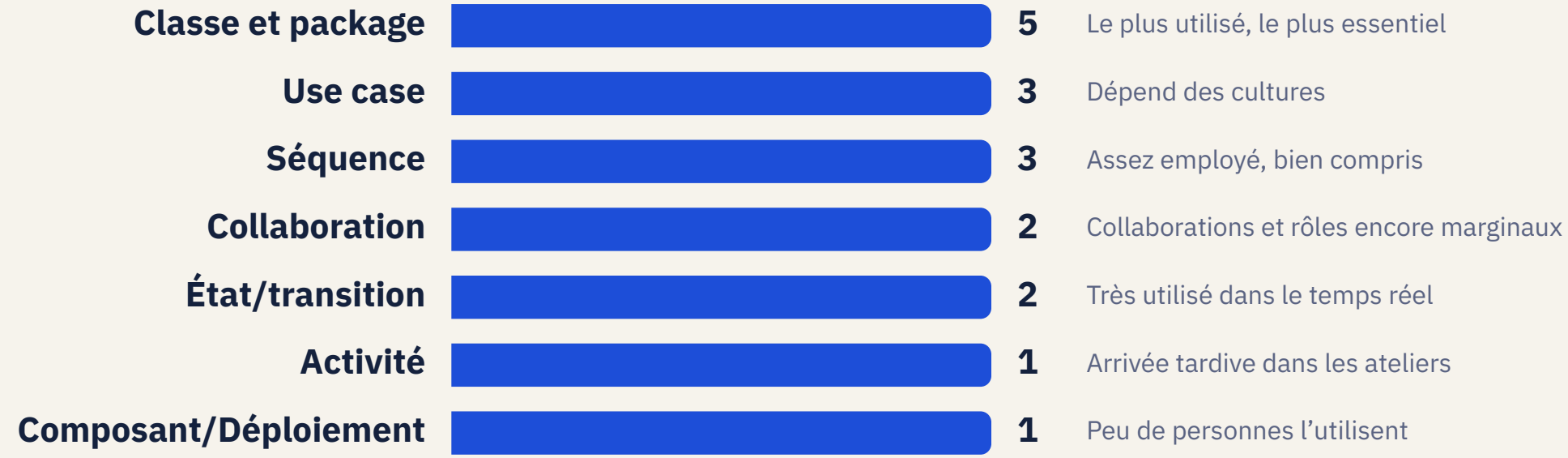
Quel diagramme montre l'ordre des messages entre le client, le système et la banque ?

- **A** Diagramme de classes
- **B** Diagramme de cas d'utilisation
- **C** Diagramme de déploiement
- **D** Diagramme de séquence

Réponse D : diagramme de séquence

Il montre qui envoie quel message à qui, dans l'ordre, de haut en bas.

Quels diagrammes UML sont les plus utilisés ?



Niveau d'utilisation dans l'industrie, de 1 à 5, tel que présenté dans le cours.

À retenir : les diagrammes de classes et de packages dominent.

MCD : les concepts de base

Entité

Objet concret ou abstrait ayant une existence propre ; la classe d'entités les regroupe.

« Dupond » est une occurrence de la classe CLIENT

Association

Lien sémantique entre entités ; la classe d'associations regroupe les liens de même type.

PROPRIETAIRE(PERSONNE, VOITURE)

Propriété

Donnée élémentaire qui caractérise une entité ou une association.

Nom, prénom, adresse pour PERSONNE

Identifiant

Propriété (ou groupe) qui identifie de façon unique chaque occurrence ; il est souligné.

N° d'immatriculation pour VOITURE

MCD : les cardinalités

MINIMUM : 0 OU 1 · MAXIMUM : 1 OU n

0,1	**1,1**	**1,n**	**0,n**
au plus 1	1 et 1 seul	au moins 1	un nombre quelconque



Une personne a 0 à n voitures ; une voitures a 1 et 1 seul propriétaire.

EN CLAIR

Lire une cardinalité en 3 étapes

ÉTAPE 1

Choisir une entité

Par exemple VOITURE.

ÉTAPE 2

Lire le couple collé à elle

Ici, (1,1).

ÉTAPE 3

Compléter la phrase

Une voiture a au minimum 1 et au maximum 1 propriétaire.

✓ **Minimum : 0 = facultatif, 1 = obligatoire. Maximum : 1 = un seul, n = plusieurs.**

QUIZ · 1 MIN

Une voiture a un et un seul propriétaire : quelle cardinalité côté VOITURE ?

A 0,1

B 1,1

C 0,n

D 1,n

Réponse B : (1,1)

Minimum 1 : toute voiture a un propriétaire. Maximum 1 : un seul.

Entités : 3 règles de qualité

RÈGLE 1

Un identifiant

Toute entité a un identifiant.

VOITURE : N° d'immatriculation. Le nom ne suffit pas pour une PERSONNE : on ajoute un N° client.

RÈGLE 2

Propriétés mono-valuées

Pour une occurrence, chaque propriété ne prend qu'une seule valeur (1FN).

Employé(Matricule, Nom, Prénom-enfant) devient Employé (Matricule, Nom) et Enfant(Matricule, Prénom-enfant).

RÈGLE 3

Dépendance à l'identifiant

Toute propriété dépend de l'identifiant, et uniquement de lui (2FN, 3FN).

Dans VOITURE, N°insee détermine Nom et Adresse : on crée l'entité PROPRIETAIRE à part.

✓ **Mono-valuée : 1FN. Dépendance à l'identifiant : 2FN et 3FN.**

Associations : 4 règles de qualité

Propriétés mono-valuées

Pour une occurrence d'association, chaque propriété ne prend qu'une seule valeur.

Dépendance aux identifiants

Chaque propriété dépend de tous les identifiants des entités liées, et d'eux seuls.

Date-permis dépend de N°insee seul : on la déplace vers PERSONNE.

Associations n-aires

Les garder au minimum : si une dépendance relie une partie des entités, on décompose.

COURS(Prof, Matière, Classe) avec N°prof → N°mat : on décompose.

Associations transitives

Une association déductible d'autres par transitivité est supprimée.

Facture–Représentant se déduit de Facture–Commande et Commande–Représentant.

EN CLAIR

Du dessin aux tables

ENTITÉ

Une feuille

VOITURE devient une table : une ligne par voiture, une colonne par propriété.

IDENTIFIANT

Un numéro unique

Le n° d'immatriculation devient la clé primaire : il identifie chaque ligne.

ASSOCIATION

Un renvoi

N°insee, dans VOITURE, renvoie à la personne : c'est la clé étrangère.

Règle d'or du 1:N : le côté « plusieurs » porte le numéro du côté « un ».

Du MCD au MLD : règles de passage

Entité	**Une table** par entité ; ses propriétés deviennent les **colonnes**, son identifiant la **clé primaire**.
1:1	La clé primaire d'une des deux tables devient **clé étrangère** dans l'autre.
1:N	La clé primaire du côté **1** devient **clé étrangère** dans la table du côté **N**.
N:M	Une **table de jonction** reprend les clés primaires des deux entités, comme clés étrangères.

Clé primaire (PK) : identifie chaque ligne. Clé étrangère (FK) : référence une PK.

Compléter le MLD

Attribut multivalué
Une nouvelle table, liée à la table parente par une clé étrangère.

Héritage
Une table par sous-type, ou une table unique regroupant les attributs.

Types de données
Un type adapté à chaque colonne : entier, texte, date…

Contraintes
Intégrité référentielle : les clés étrangères assurent la cohérence.

Indices
Sur les colonnes souvent interrogées, pour accélérer les recherches.

Documentation
Le sens de chaque table, colonne, clé et contrainte.

✓ **Normalisation : éviter les redondances et les anomalies.**

Le MCD de PERSONNE et VOITURE



Au passage au MLD : chaque entité devient une table ; l'association disparaît dans une clé étrangère.

Exemple : PERSONNE et VOITURE



MCD : PERSONNE (0,n) — PROPRIETAIRE — (1,1) VOITURE

L'association PROPRIETAIRE disparaît : elle devient la clé étrangère N°insee dans VOITURE.

QUIZ · 1 MIN

Dans la table VOITURE, la colonne N°insee (le propriétaire) est une…

- **A** clé primaire
- **B** clé étrangère
- **C** entité
- **D** cardinalité

Réponse B : clé étrangère

Elle contient la clé primaire de PERSONNE : c'est ainsi que VOITURE renvoie à son propriétaire.

À VOUS · 10 MIN

La bibliothèque universitaire

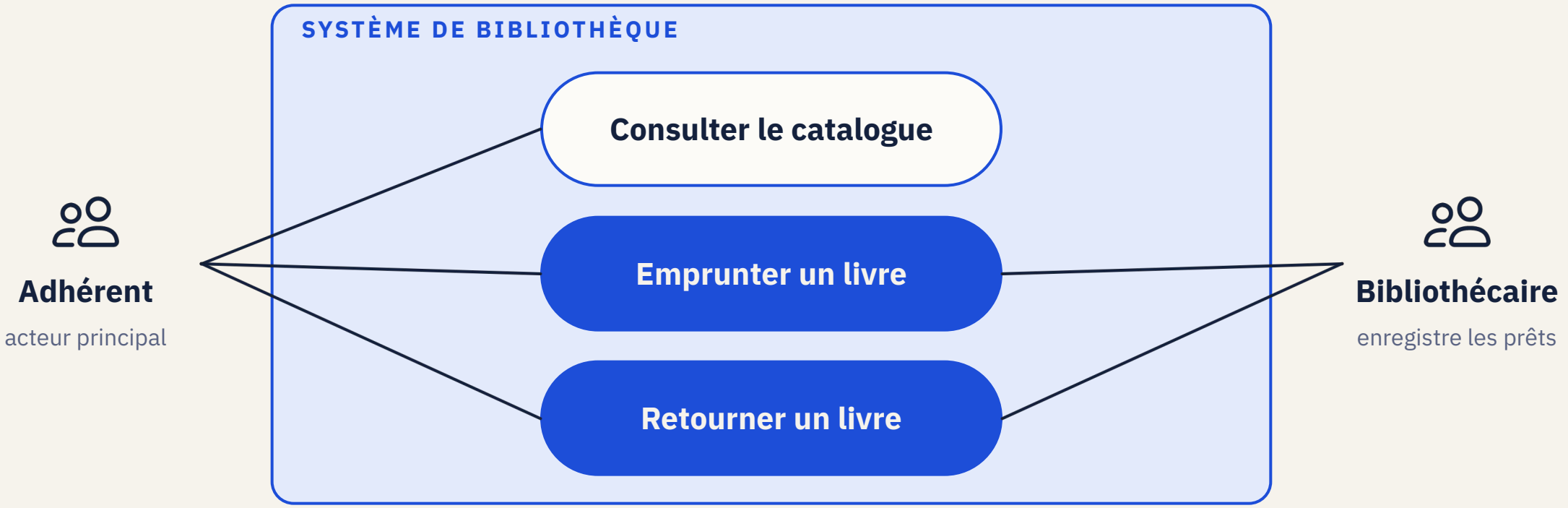
ÉNONCÉ

Des adhérents empruntent des livres à la bibliothèque universitaire. Un adhérent peut emprunter plusieurs livres, et un livre peut être emprunté par plusieurs adhérents, à des dates différentes. On enregistre la date d'emprunt et la date de retour.

1. Cas d'utilisation : quels acteurs, quels cas ?
2. MCD : entités, association, cardinalités, propriétés.
3. MLD : tables, clés primaires et clés étrangères.

Travaillez en binôme pendant 10 minutes ; la correction suit, étape par étape.

Correction 1 : les cas d’utilisation



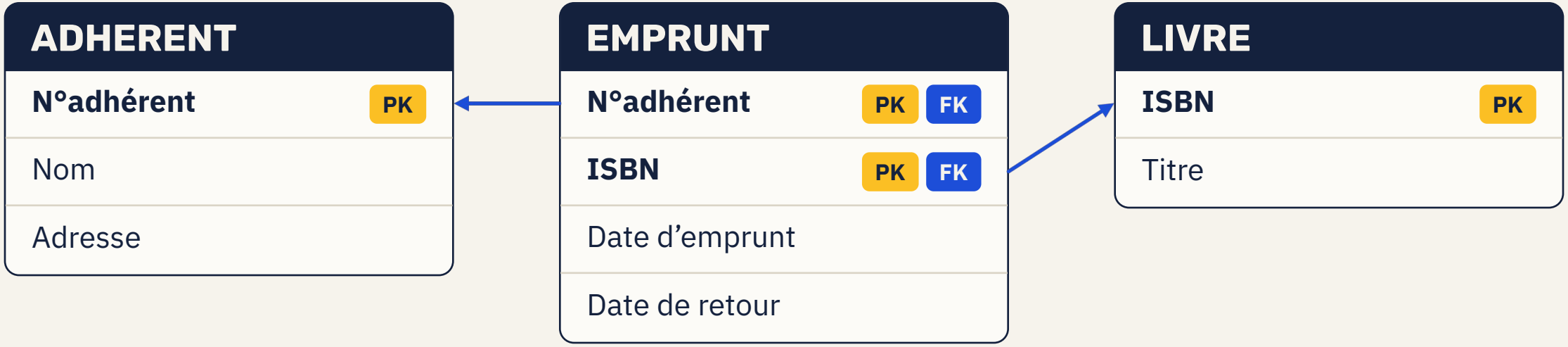
Un cas d’utilisation est un but d’un acteur : consulter, emprunter, retourner.

Correction 2 : le MCD



Une association N:M peut porter des propriétés : ici, les deux dates.

Correction 3 : le MLD



L’association N:M devient une table de jonction : sa clé primaire est formée des deux clés étrangères.

POUR ALLER PLUS LOIN

Et si on devait construire ce logiciel ?

1. Quelle méthode choisir pour ce projet ? Justifiez avec la stabilité des besoins et le risque.

2. Citez un besoin non fonctionnel de ce système (performance, sécurité, ergonomie…).

3. À quelle étape rédige-t-on le cahier des charges, et quels modèles produit-on ensuite ?

Pas de réponse unique : l'important est de justifier son choix.

Révision

Cinq questions pour tout reprendre

Systèmes · Architecture · Méthodes · Étapes · MCD

RÉVISION · 1/5

Dans une pizzeria, le gérant qui fixe les prix relève de quel sous-système ?

A **Pilotage (SP)**

B **Opérant (SO)**

C **Information (SI)**

D **Aucun des trois**

Réponse A : pilotage (SP)

Le pilotage décide et fixe les objectifs ; la cuisine est l'opérant, la caisse relève de l'information.

RÉVISION · 2/5

Quel est l’objectif de l’architecture d’entreprise ?

A **Écrire le code des applications**

B **Choisir un langage de programmation**

C **Dessiner les écrans d’une application**

D **Aligner l’informatique sur les objectifs de l’entreprise**

Réponse D : aligner l’informatique sur les objectifs

L’architecture d’entreprise relie la stratégie, le métier et la technologie.

RÉVISION · 3/5

Quel modèle place l'analyse des risques au centre de chaque tour ?

A **Cascade**

B **Incrémental**

C **Spirale**

D **Modèle en V**

Réponse C : spirale

Chaque tour commence par les objectifs, puis l'analyse des risques, avant de développer et de vérifier.

RÉVISION · 4/5

Quelle étape répond à : comment le construire, avec quelle technologie ?

A **Conception**

B **Analyse**

C **Test**

D **Étude préalable**

Réponse A : conception

L’analyse dit quoi, sans technologie ; la conception dit comment, pour une technologie précise.

RÉVISION · 5/5

Dans le MCD, « Dupond » est… de la classe d'entités CLIENT.

A une propriété

B une occurrence

C un identifiant

D une association

Réponse B : une occurrence

La classe d'entités regroupe les occurrences : Dupond est un exemple concret de CLIENT.

Lexique 1/3 : entreprise et architecture

Terme	En clair
Système	Éléments en interaction, organisés en fonction d’un but.
SP · pilotage	Décide, planifie et contrôle : fixe les objectifs.
SI · information	Fait circuler, mémorise et traite l’information.
SO · opérant	Cœur de métier : transforme et produit.
Système d’information	Toute l’information de l’entreprise, informatisée ou non.
Système informatique	La partie du SI qui tourne sur des ordinateurs.
ESA	Architecture d’entreprise : aligne l’informatique sur la stratégie.
Architecture logicielle	Organisation interne d’un logiciel : composants, règles.
Patron d’architecture	Organisation éprouvée : couches, MVC, microservices.

Lexique 2/3 : méthodes et développement

Terme	En clair
Cycle de vie	Étapes d’un logiciel, du besoin à la livraison.
Itération	Petit cycle de travail complet qui ajoute un incrément.
Incrément	Morceau fonctionnel ajouté au produit.
Sprint	Cycle Scrum de 2 semaines à 1 mois, qui livre un incrément.
Backlog du produit	Liste priorisée des besoins du client.
User story	Fonctionnalité décrite du point de vue de l’utilisateur.
Cahier des charges	Ce que le client attend, par écrit : livrable de l’étude préalable.
Exigence fonctionnelle	Ce que le système fait : réserver, consulter, enregistrer.
Exigence non fonctionnelle	Qualité attendue : performance, sécurité, ergonomie.

Lexique 3/3 : modélisation et données

Terme	En clair
Acteur	Personne ou système externe qui interagit avec le système.
Cas d’utilisation	Un but du système, motivé par le besoin d’un acteur.
Classe · objet	Un type de choses · un exemple concret de ce type.
MCD	Plan conceptuel des données, sans parler de technologie.
Entité · occurrence	Un type de chose (CLIENT) · un exemple (Dupond).
Association	Lien entre entités (PROPRIETAIRE entre PERSONNE et VOITURE).
Cardinalité	Couple (min, max) : combien de liens pour une occurrence.
MLD	Le MCD traduit en tables.
Clés primaire · étrangère	Identifie une ligne · renvoie à la clé d’une autre table.

Merci

Des questions ?

1 · Le système d’entreprise
2 · L’architecture
3 · La mise en œuvre
4 · Les activités de développement